

清华大学本科生考试试题纸		
考试课程：操作系统	时间：2023年11月02日上午09:50~11:20	B卷
姓名：_____	班级：_____	学号：_____
答卷注意事项:	1. 答题前在答卷纸每一页左上角写明姓名、班级、学号、页码和总页数。	
	2. 在答卷纸上答题时, 要写明题号, 不必抄题。	
	3. 答题时, 要书写清楚和整洁。	
	4. 请注意回答所有试题。本试卷有15个小题，共14页。	
	5. 完成答卷后，请将试题和答卷一起交回。	

一、对错题（20分）

- 1. ☐ 进程切换既可以在用户态进行，也可以在内核态进行。
- 2. ☐ Linux 中系统调用 exec 会创建新的虚拟地址空间。
- 3. ☐ Linux 中系统调用 fork 会创建新的虚拟地址空间。
- 4. ☐ 父进程先于子进程退出会导致子进程变为僵尸进程。
- 5. ☐ 系统调用过程中的参数和返回值传递是通过内核栈进行的。
- 6. ☐ RISC-V 处理器在用户态 (U-mode) 时执行 ecall 指令，那么下一条指令一定在内核态 (S-mode)中执行。
- 7. ☐ 处理执行指令中的缺页异常时，可以根据 sepc 寄存器的值唯一确定缺页的虚拟页号。
- 8. ☐ 在 RISC-V 指令架构中，时钟相关寄存器 mtimecmp 通过 csrr/w 指令进行读写。
- 9. ☐ 在 RISC-V 指令架构中，已知处理器支持 M，S，U 三个特权级，U 特权级下的异常一定会由 M 特权级进行处理。
- 10. ☐ rCore 和 uCore 的内核运行在 RISC-V 的 S 特权级。

二、简答题

11. （8分）

- 1. RISC-V 指令集 RV32I 支持加载有符号和无符号字节和半字（lb，lbu，lh，lhu）和存储字节和半字（sb，sh）。用户态的一个加载指令，单次执行最多造成几次 Load page fault 异常？为什么？
- 2. 部分指令集架构中包含有向量计算，向量访存指令通常包含有一个 Strided 模式，用于按照固定的地址间隔加载元素。考虑一个 512 位的向量实现，保存有 8 个 64 位整数，在这一实

现上执行 Strided 加载访存，其功能伪代码如下：

```
// vd 是向量寄存器, rs1, rs2 是标量(x0-x31)寄存器。
// vlse64.v vd, (rs1), rs2
for(int i = 0; i < 8; ++i) {
    vd[i] = *(rs1 + rs2 * i)
}
```

假设内核对缺页的处理都是添加缺失的页表项目，并不移除任何页表项目，那么一条 vlse64.v 指令最多造成几次 Load page fault?请构造一个触发三次 Load page fault 的例子，给出 rs1, rs2 的值，以及相关的页表项目的有效性(只需给出叶节点)。

3. vlse64.v 指令除了 Load page fault 以外还可能造成什么异常？

12. (12分)

请简要描述下面7种处理机调度算法的就绪队列的排队判断条件或公式。

1. 先来先服务调度算法FCFS
2. 短作业优先调度算法SJF
3. 最高响应比优先算法HRRN
4. 速率单调调度算法(RM, Rate Monotonic)
5. 最早截止时间优先算法 (EDF, Earliest Deadline First)
6. 最低松弛度优先算法 (LLF)

13. (20分)

1. 请简要描述CFS调度算法的基本工作原理。
2. 在CFS调度算法中，所有进程的虚拟时间vruntime增长速度宏观上是同时推进的，可用虚拟时间vruntime作为就绪进程队列的排队顺序。假定每个时间片的长度为一个时间单位；优先数表示CPU时间占比。请给出下面场景下CFS调度算法在前15个时间单位中，各进程的虚拟时间变化情况和处理机调度顺序。

时间	进程A	进程B	进程C	进程D	进程E	调度顺序
优先数	2	3	4	2	3	
进入就绪时间	0	0.1	0.2	0.3	5.1	
时间	虚拟时间					调度顺序
0	0					A
1						
2						
3						
.....						

14. (22分)

有一台假想的计算机，页大小（page size）为32 Bytes，支持32KB的虚拟地址空间（virtual address space），有4KB的物理内存空间（physical memory），采用二级页表，一个页目录项（page directory entry，PDE）大小为1 Byte，一个页表项（page-table entries PTEs）大小为1 Byte，1个页目录表大小为32 Bytes，1个页表大小为32 Bytes。页目录基址寄存器（page directory base register，PDBR）保存了页目录表的物理地址（按页对齐）。

PTE格式（8 bit）：

VALID PFN6 ... PFN0

PDE格式（8 bit）：

VALID PT6 ... PT0

其中

VALID==1表示，表示映射存在；VALID==0表示，表示映射不存在。 PFN6..0:页帧号 PT6..0:页表的物理基址>>5

页目录基址寄存器（page directory base register，PDBR）保存了页目录表的物理地址（按页对齐），其值为0x220（十进制为544）。

页目录表的内容如下：(其中数值都是十六进制)

```
page 11: da f7 f2 a8 96 c5 9d 94 c8 b9 7f c4 98 e5 7f 7f d3 a1 82 8f a6 fb
bf f0 7f 84 d2 a0 88 80 c9 92
```

1. 请问在这台假想计算机的虚拟地址中，两级页号和页内偏移各占多少位？
2. 这台假想的计算机上执行访存指令的过程中可能出现缺页异常。请简要描述缺页异常的处理过程。
3. 已知这台假想的计算机上执行访存指令时虚拟地址 `0x2c00` 经地址转换后得到的物理地址为 `0xae0`。请回答对应页目录项的序号 (pde index) 和内容 (pde contents)，页表项是否存在？如果对应页表项存在，请回答对应页表项的序号 (pte index) 和内容 (pte contents)；如果对应页表项不存在，则虚拟地址也不在内存中，需要分配两个空闲物理页面，物理地址为 `0x420` 的作为第二级页表，物理地址为 `0x6a0` 作为虚拟地址 `0x2c00` 所在页面) 请回答更新后的页目录项内容 (pde contents)、对应页表项的序号 (pte index) 和内容 (pte contents)。
4. 已知这台假想的计算机上执行访存指令时虚拟地址 `0x3168` 经地址转换后得到的物理地址为 `0x6a8`。请回答对应页目录项的序号 (pde index) 和内容 (pde contents)，页表项是否存在？如果对应页表项存在，请回答对应页表项的序号 (pte index) 和内容 (pte contents)；如果对应页表项不存在，则虚拟地址也不在内存中，需要分配两个空闲物理页面，物理地址为 `0x420` 的作为第二级页表，物理地址为 `0x6a0` 作为虚拟地址 `0x2c00` 所在页面) 请回答更新后的页目录项内容 (pde contents)、对应页表项的序号 (pte index) 和内容 (pte contents)。

15. (18分)

阅读ch2版本下的代码，并回答下列几个问题。请同学们根据自己实验选择完成对应版本的回答即可。

uCore版本

trap.h

```
#ifndef TRAP_H
#define TRAP_H

#include "types.h"

struct trapframe {
    /* 0 */ uint64 kernel_satp; // kernel page table
    /* 8 */ uint64 kernel_sp; // top of process's kernel stack
    /* 16 */ uint64 kernel_trap; // usertrap()
    /* 24 */ uint64 epc; // saved user program counter
    /* 32 */ uint64 kernel_hartid; // saved kernel tp
    /* 40 */ uint64 ra;
    /* 48 */ uint64 sp;
};
```

```

/* 56 */ uint64 gp;
/* 64 */ uint64 tp;
/* 72 */ uint64 t0;
/* 80 */ uint64 t1;
/* 88 */ uint64 t2;
/* 96 */ uint64 s0;
/* 104 */ uint64 s1;
/* 112 */ uint64 a0;
/* 120 */ uint64 a1;
/* 128 */ uint64 a2;
/* 136 */ uint64 a3;
/* 144 */ uint64 a4;
/* 152 */ uint64 a5;
/* 160 */ uint64 a6;
/* 168 */ uint64 a7;
/* 176 */ uint64 s2;
/* 184 */ uint64 s3;
/* 192 */ uint64 s4;
/* 200 */ uint64 s5;
/* 208 */ uint64 s6;
/* 216 */ uint64 s7;
/* 224 */ uint64 s8;
/* 232 */ uint64 s9;
/* 240 */ uint64 s10;
/* 248 */ uint64 s11;
/* 256 */ uint64 t3;
/* 264 */ uint64 t4;
/* 272 */ uint64 t5;
/* 280 */ uint64 t6;
};

enum Exception {
    InstructionMisaligned = 0,
    InstructionAccessFault = 1,
    IllegalInstruction = 2,
    Breakpoint = 3,
    LoadMisaligned = 4,
    LoadAccessFault = 5,
    StoreMisaligned = 6,
    StoreAccessFault = 7,
    UserEnvCall = 8,
    SupervisorEnvCall = 9,
    MachineEnvCall = 11,
    InstructionPageFault = 12,
    LoadPageFault = 13,
    StorePageFault = 15,

```

```

};

enum Interrupt {
    UserSoft = 0,
    SupervisorSoft,
    UserTimer = 4,
    SupervisorTimer,
    UserExternal = 8,
    SupervisorExternal,
};

void trap_init();
void usertrapret(struct trapframe *trapframe, uint64 kstack);

#endif // TRAP_H

```

trap.c

```

#include "trap.h"
#include "defs.h"
#include "loader.h"
#include "syscall.h"

extern char trampoline[], uservec[], boot_stack_top[];
extern void *userret(uint64);

void trap_init(void)
{
    w_stvec((uint64)uservec & ~0x3);
}

void usertrap(struct trapframe *trapframe)
{
    if ((r_sstatus() & SSTATUS_SPP) != 0)
        panic("usertrap: not from user mode");

    uint64 cause = r_scause();
    if (cause == UserEnvCall) {
        trapframe->epc += 4;
        syscall();
        return usertrapret(trapframe, (uint64)boot_stack_top);
    }
    switch (cause) {
        case StoreMisaligned:
        case StorePageFault:

```

```

    case LoadMisaligned:
    case LoadPageFault:
    case InstructionMisaligned:
    case InstructionPageFault:
        errorf("%d in application, bad addr = %p, bad instruction = %p, core
"
                "dumped.",
                cause, r_stval(), trapframe->epc);
        break;
    case IllegalInstruction:
        errorf("IllegalInstruction in application, epc = %p, core dumped.",
                trapframe->epc);
        break;
    default:
        errorf("unknown trap: %p, stval = %p sepc = %p", r_scause(),
                r_stval(), r_sepc());
        break;
}
infof("switch to next app");
run_next_app();
printf("ALL DONE\n");
shutdown();
}

//
// return to user space
//
void usertrapret(struct trapframe *trapframe, uint64 kstack)
{
    trapframe->kernel_satp = r_satp(); // kernel page table
    trapframe->kernel_sp = kstack + PGSIZE; // process's kernel stack
    trapframe->kernel_trap = (uint64)usertrap;
    trapframe->kernel_hartid = r_tp(); // hartid for cpuid()

    w_sepc(trapframe->epc);

    uint64 x = r_sstatus();
    x &= ~SSTATUS_SPP; // clear SPP to 0 for user mode
    x |= SSTATUS_SPIE; // enable interrupts in user mode
    w_sstatus(x);

    userret((uint64)trapframe);
}

```

trampoline.S

```

    # code to switch between user and kernel space.
    #
    # this code is mapped at the same virtual address
    # (TRAMPOLINE) in user and kernel space so that
    # it continues to work when it switches page tables.
    #
    # kernel.ld causes this to be aligned
    # to a page boundary.
    #

.section .text
.globl trampoline
trampoline:
.align 4
.globl uservec
uservec:
    # trap.c sets stvec to point here, so
    # traps from user space start here

    # (1)start=====
    # swap a0 and sscratch
    csrrw a0, sscratch, a0
    # (1)end=====
    # (4)
    # (2) start=====
    sd ra, 40(a0)
    sd sp, 48(a0)
    sd gp, 56(a0)
    sd tp, 64(a0)
    sd t0, 72(a0)
    sd t1, 80(a0)
    sd t2, 88(a0)
    sd s0, 96(a0)
    sd s1, 104(a0)
    sd a1, 120(a0)
    sd a2, 128(a0)
    sd a3, 136(a0)
    sd a4, 144(a0)
    sd a5, 152(a0)
    sd a6, 160(a0)
    sd a7, 168(a0)
    sd s2, 176(a0)
    sd s3, 184(a0)
    sd s4, 192(a0)
    sd s5, 200(a0)

```



```

sd s6, 208(a0)
sd s7, 216(a0)
sd s8, 224(a0)
sd s9, 232(a0)
sd s10, 240(a0)
sd s11, 248(a0)
sd t3, 256(a0)
sd t4, 264(a0)
sd t5, 272(a0)
sd t6, 280(a0)
# (2) end=====

# (3)start=====
csrr t0, sscratch
sd t0, 112(a0)

csrr t1, sepc
sd t1, 24(a0)
# (3)end=====

ld sp, 8(a0)
ld tp, 32(a0)
ld t1, 0(a0)
# csrwr satp, t1
# sfence.vma zero, zero
ld t0, 16(a0)
jr t0

```

```

.globl userret

```

```

userret:
# userret(TRAPFRAME, pagetable)
# switch from kernel to user.
# usertrapret() calls here.

ld t0, 112(a0)
csrr sscratch, t0

ld ra, 40(a0)
ld sp, 48(a0)
ld gp, 56(a0)
ld tp, 64(a0)
ld t0, 72(a0)
ld t1, 80(a0)
ld t2, 88(a0)
ld s0, 96(a0)
ld s1, 104(a0)

```

```

ld a1, 120(a0)
ld a2, 128(a0)
ld a3, 136(a0)
ld a4, 144(a0)
ld a5, 152(a0)
ld a6, 160(a0)
ld a7, 168(a0)
ld s2, 176(a0)
ld s3, 184(a0)
ld s4, 192(a0)
ld s5, 200(a0)
ld s6, 208(a0)
ld s7, 216(a0)
ld s8, 224(a0)
ld s9, 232(a0)
ld s10, 240(a0)
ld s11, 248(a0)
ld t3, 256(a0)
ld t4, 264(a0)
ld t5, 272(a0)
ld t6, 280(a0)

csrrw a0, sscratch, a0

sret

```

问题：

1. 执行(1)处代码时，CPU的特权级是U态，还是S态，还是M态？
2. 发生中断/异常后，CPU会跳转到uservec处开始执行。为什么？请说明这一过程中的硬件行为和软件行为。（答案请控制在2句话，每句20个汉字以内）
3. (1)处的代码含义是将sscratch和a0的值交换。交换之后的a0是什么含义？（答案请控制在20个汉字以内）
4. (2)处的代码完成了什么功能？（答案请控制在20个汉字以内）
5. (3)处的代码完成了什么功能？（答案请控制在20个汉字以内）这段代码不能放在(4)处吗？为什么？（答案请控制在30个汉字以内）

rCore版本

trap/mod.rs

```

mod context;

use crate::batch::run_next_app;
use crate::syscall::syscall;

```

```

use core::arch::global_asm;
use riscv::register::{
    mtvec::TrapMode,
    scause::{self, Exception, Trap},
    stval, stvec,
};

global_asm!(include_str!("trap.S"));

pub fn init() {
    extern "C" {
        fn __alltraps();
    }
    unsafe {
        stvec::write(__alltraps as usize, TrapMode::Direct);
    }
}

#[no_mangle]
/// handle an interrupt, exception, or system call from user space
pub fn trap_handler(cx: &mut TrapContext) -> &mut TrapContext {
    let scause = scause::read(); // get trap cause
    let stval = stval::read(); // get extra value
    match scause.cause() {
        Trap::Exception(Exception::UserEnvCall) => {
            cx.sepc += 4;
            cx.x[10] = syscall(cx.x[17], [cx.x[10], cx.x[11], cx.x[12]]) as
usize;
        }
        Trap::Exception(Exception::StoreFault) |
Trap::Exception(Exception::StorePageFault) => {
            println!("[kernel] PageFault in application, kernel killed
it.");
            run_next_app();
        }
        Trap::Exception(Exception::IllegalInstruction) => {
            println!("[kernel] IllegalInstruction in application, kernel
killed it.");
            run_next_app();
        }
        _ => {
            panic!(
                "Unsupported trap {:?}, stval = {:#x}!",
                scause.cause(),
                stval
            );
        }
    }
}

```

```

    }
}
cx
}

pub use context::TrapContext;

```

trap/context.rs

```

use riscv::register::sstatus::{self, Sstatus, SPP};
/// Trap Context
#[repr(C)]
pub struct TrapContext {
    /// general regs[0..31]
    pub x: [usize; 32],
    /// CSR sstatus
    pub sstatus: Sstatus,
    /// CSR sepc
    pub sepc: usize,
}

impl TrapContext {
    /// set stack pointer to x_2 reg (sp)
    pub fn set_sp(&mut self, sp: usize) {
        self.x[2] = sp;
    }
    /// init app context
    pub fn app_init_context(entry: usize, sp: usize) -> Self {
        let mut sstatus = sstatus::read(); // CSR sstatus
        sstatus.set_spp(SPP::User); //previous privilege mode: user mode
        let mut cx = Self {
            x: [0; 32],
            sstatus,
            sepc: entry, // entry point of app
        };
        cx.set_sp(sp); // app's user stack pointer
        cx // return initial Trap Context of app
    }
}

```

trap/trap.S

```

.altmacro
.macro SAVE_GP n

```

```

        sd x\n, \n*8(sp)
    .endm
    .macro LOAD_GP n
        ld x\n, \n*8(sp)
    .endm

    .section .text
    .globl __alltraps
    .globl __restore
    .align 2
__alltraps:
    # (1)start=====
    csrrw sp, sscratch, sp
    # (1)end=====
    # (4)
    # (2)start=====
    addi sp, sp, -34*8
    sd x1, 1*8(sp)
    sd x3, 3*8(sp)
    .set n, 5
    .rept 27
        SAVE_GP %n
        .set n, n+1
    .endr
    # (2)end=====

    # (3)start=====
    csrr t0, sstatus
    csrr t1, sepc
    sd t0, 32*8(sp)
    sd t1, 33*8(sp)
    # (3)end=====

    csrr t2, sscratch
    sd t2, 2*8(sp)

    mv a0, sp
    call trap_handler

__restore:
    mv sp, a0

    ld t0, 32*8(sp)
    ld t1, 33*8(sp)
    ld t2, 2*8(sp)
    csrw sstatus, t0
    csrw sepc, t1

```

```

    csrw sscratch, t2

    ld x1, 1*8(sp)
    ld x3, 3*8(sp)
    .set n, 5
    .rept 27
        LOAD_GP %n
        .set n, n+1
    .endr

    addi sp, sp, 34*8

    csrrw sp, sscratch, sp
    sret

```

问题：

1. 执行(1)处代码时，CPU的特权级是U态，还是S态，还是M态？
2. 发生中断/异常后，CPU会跳转到__alltraps处开始执行。为什么？请说明这一过程中的硬件行为和软件行为。（答案请控制在2句话，每句20个汉字以内）
3. (1)处的代码含义是将sscratch和sp的值交换。交换之后的sp是什么含义？（答案请控制在20个汉字以内）
4. (2)处的代码完成了什么功能？（答案请控制在20个汉字以内）
5. (3)处的代码完成了什么功能？（答案请控制在20个汉字以内）这段代码不能放在(4)处吗？为什么？（答案请控制在30个汉字以内）